

VINI VICI VIDI

PREMIUM SILVER COLLECTION

RECOMMENDED TECHNOLOGY STACK + SECURITY ARCHITECTURE

A production baseline for the cinematic luxury storefront, 3D experience, guest checkout, Razorpay payments, live pricing, inventory, WhatsApp and Shopify-like order workflow.

Core principle: the visual experience must remain highly custom, while the commerce backend must remain server-authoritative and security-first.

Prepared as a single technical reference for implementation and handoff.

1. Recommended Production Stack

Layer	Choice	Purpose
Framework	Next.js + React + TypeScript	Custom storefront, server layer and routing in one application.
Rendering	Next.js App Router	Fast page delivery plus dynamic commerce surfaces.
Styling	Tailwind CSS + custom CSS	Rapid iteration without forcing a template aesthetic.
3D	Three.js + React Three Fiber + Drei	Interactive silver products, environments and hero scenes.
3D post-processing	React Three Fiber postprocessing	Depth, bloom, vignette and cinematic treatment where useful.
Animation	GSAP + ScrollTrigger	Hero timelines, pinned scenes, horizontal movement and scroll choreography.
Smooth scroll	Lenis	Inertial, controlled scrolling for the premium experience.
UI motion	Framer Motion	Menus, drawers, dialogs and micro-interactions.
State	Zustand	Guest cart and lightweight client state.
Database	PostgreSQL	Products, prices, inventory, promotions, orders and payments.
Backend platform	Supabase	PostgreSQL, storage, realtime and admin authentication.
ORM	Drizzle ORM	Typed and maintainable database access.
Admin auth	Supabase Auth	Admin-only authentication. No customer accounts.
Media	Supabase Storage	Product image and media storage.
Payments	Razorpay	Required payment gateway.
Realtime	Supabase Realtime	Live storefront synchronization for price/stock data.
WhatsApp	WhatsApp link / business integration	Customer communication and product enquiries.
Deployment	Vercel	Production hosting for Next.js.
Repository	GitHub	Version control and CI/CD workflow.
Monitoring	Sentry	Production error monitoring and debugging.

2. Recommended System Architecture

```
CUSTOMER BROWSER
|
v
+-----+
| NEXT.JS |
| App Router / API |
+-----+
|
+-----+-----+
| | |
v v v
Premium UI 3D Engine Commerce UI
Tailwind Three.js Zustand Cart
GSAP R3F / Drei Guest Checkout
Lenis Effects Product Pages
| | |
+-----+-----+
|
v
SERVER LAYER
|
+-----+
| | |
v v v
Products Orders Payments
| | |
+-----+
|
v
POSTGRESQL / SUPABASE
| | |
Inventory Realtime Storage
|
v
RAZORPAY
|
v
WEBHOOKS
|
v
VERIFIED ORDER FLOW
```

Architecture rule: the browser is treated as untrusted. Price, stock, discounts, order totals and payment state are determined or verified on trusted server-side paths.

3. Visual / Interaction Stack

Experience	Implementation
Mega cinematic entry	GSAP master timeline + Three.js scene + staged light reveal
Flashlight effect	Pointer-driven softened spotlight / shader mask with inertia
Liquid chrome background	Three.js procedural geometry/materials and restrained reflective movement
Hero product	React Three Fiber + physically based metallic material + animated camera
Scroll choreography	GSAP ScrollTrigger + Lenis
Editorial typography	CSS transforms + GSAP timelines + layered composition
Product gallery	Custom pinned / horizontal / asymmetric compositions
UI transitions	Framer Motion for menus, cart drawers and dialogs

4. Commerce Architecture

```
Customer
|
v
Product Page -> Cart -> Guest Checkout
|
v
Create Razorpay Order
|
v
Razorpay Checkout
|
v
Server Verification
|
+-----+-----+
||
v v
Create/confirm Order Payment Event Log
|
v
Inventory Transaction
|
v
Admin Dashboard
```

- Guest checkout: customers do not need accounts or registration.
- Razorpay: create payment orders on the server; verify payment and webhook events on the server before marking orders as paid.
- Order snapshot: store the exact purchase-time price, discount, quantity and totals on the order.
- Inventory: update stock through backend-controlled transactions so two simultaneous buyers cannot both consume the same last unit.

5. Real-Time Pricing and Inventory

- Admin updates price in PostgreSQL.
- Realtime notification reaches active storefront sessions where appropriate.
- Checkout re-checks current price and stock on the server.
- Razorpay amount is generated from the server-calculated total.
- Historical orders retain their original price even after later price changes.
- Low-stock messages such as “ONLY 3 LEFT” come from actual inventory data and configurable thresholds.

6. Security Principles - Non-Negotiable

Principle	Requirement
Browser is untrusted	Never trust frontend values for price, inventory, discount, order total or payment state.
Secrets stay server-side	Razorpay secret keys, database credentials and privileged Supabase keys never ship to the browser.
Admin is authenticated + authorized	Protect the admin application and every admin API, not just the UI route.
Database enforces access	Use PostgreSQL/Supabase Row Level Security for sensitive records.
Payments are verified	Use server-side Razorpay verification and verified webhook handling.
Input is validated	Validate checkout, quantities, coupons, IDs, addresses and admin input with strict schemas.
APIs are rate-limited	Protect checkout, payment, admin, search and contact endpoints against abuse.
Uploads are restricted	Accept only approved image/media types and reasonable size/dimension limits.
Audit important admin actions	Record price, stock, promotion and order-status changes.
Backups exist	Maintain recoverable backups of critical commerce data.

7. Razorpay Security Flow

```
Frontend -> request checkout
|
v
Server calculates current total
|
v
Server creates Razorpay order
|
v
Razorpay Checkout
|
v
Razorpay payment + webhook events
|
v
Server verifies trusted payment data
|
+-----> Payment record
|
+-----> Order confirmation
|
+-----> Inventory update
|
+-----> Admin order status
```

Never implement: “frontend says payment successful, therefore order is paid.” That path is not an acceptable source of truth.

8. Admin Security

```
/admin/dashboard
/admin/products
/admin/inventory
/admin/pricing
/admin/promotions
/admin/orders
/admin/content
/admin/settings
```

- Protect every admin route with authentication and role checks.
- Protect every admin API independently; never depend on hidden frontend buttons.
- Use least privilege for database and service credentials.
- Keep an audit trail for sensitive actions.
- Use strong session/cookie configuration and HTTPS in production.

9. API and Abuse-Prevention Checklist

Surface	Protection
/api/checkout	Schema validation, rate limiting, server-side price/stock calculation
/api/orders	Authorization, ownership checks, server-side state transitions
/api/payment/*	Server-side verification, signature/webhook checks, idempotency where applicable
/api/admin/*	Admin authentication + authorization + audit logging
/api/search	Rate limiting, bounded query size, pagination
/api/contact	Rate limiting, spam controls, input validation
Uploads	MIME/extension/size validation, safe storage and serving
Database	Parameterized queries / ORM, RLS, least privilege
Frontend	No private secrets, safe HTML rendering, minimal third-party scripts

10. Environment and Secret Management

Example server-only environment values

RAZORPAY_KEY_ID=...

RAZORPAY_KEY_SECRET=...

DATABASE_URL=...

SUPABASE_SERVICE_ROLE_KEY=...

SENTRY_DSN=...

Do not commit .env files.

Do not use NEXT_PUBLIC_ for private secrets.

Rotate any credential that is accidentally exposed.

11. Security Testing Before Launch

- ☐ Attempt to change a product price from the browser.
- ☐ Attempt to submit a negative, huge or otherwise invalid quantity.
- ☐ Attempt to apply an unauthorized discount or promotion.
- ☐ Attempt to purchase more units than actual inventory.
- ☐ Attempt to access another customer/order record.
- ☐ Attempt to call admin APIs without authentication.
- ☐ Attempt to fake a successful Razorpay payment.
- ☐ Attempt to invoke the Razorpay webhook endpoint with forged data.
- ☐ Attempt to upload unsupported or oversized files.
- ☐ Attempt to abuse search, checkout or contact endpoints with repeated requests.
- ☐ Verify historical orders remain unchanged after product price updates.

12. Performance + Security Balance

- Optimize AVIF/WebP images, 3D textures and models.
- Lazy-load non-critical media and code-split heavy 3D experiences.
- Provide mobile and reduced-motion fallbacks.
- Limit external scripts and dependencies to those that materially help the product.
- Use security headers carefully so payment and 3D integrations continue to work.

13. Recommended Project Structure

```
vini-vici-vidi/  
├── app/  
│   ├── (store)/  
│   ├── admin/  
│   └── api/  
├── components/  
│   ├── luxury/  
│   ├── product/  
│   ├── cart/  
│   ├── checkout/  
│   └── admin/  
├── three/  
│   ├── scenes/  
│   ├── materials/  
│   ├── environments/  
│   └── effects/  
├── animations/  
│   ├── hero/  
│   ├── scroll/  
│   └── transitions/  
├── lib/  
│   ├── supabase/  
│   ├── razorpay/  
│   ├── pricing/  
│   ├── inventory/  
│   ├── orders/  
│   └── whatsapp/  
├── db/  
├── public/  
└── styles/
```

14. Final Technology Decision

NEXT.JS
REACT
TYPESCRIPT
TAILWIND CSS + CUSTOM CSS
THREE.JS
REACT THREE FIBER
DREI
GSAP
SCROLLTRIGGER
LENIS
FRAMER MOTION
ZUSTAND
SUPABASE
POSTGRESQL
DRIZZLE
RAZORPAY
WHATSAPP
VERCEL
GITHUB
SENTRY

Recommended baseline: use this stack unless a later implementation constraint provides a concrete reason to replace a component. Keep the front end art-directed and the commerce layer server-authoritative.

15. Five Highest-Priority Security Controls

- Server-side price, stock, discount and order-total validation.
- Razorpay server-side payment verification and trusted webhook handling.
- Supabase Row Level Security plus strict admin authorization.
- Secret isolation through server-only environment variables and credential rotation.
- Input validation, rate limiting, audit logging and security-focused pre-launch testing.

The visual system may be extraordinary, but the production system underneath must remain ordinary in one crucial sense: predictable, validated, observable and secure.